

In-Depth Analysis of Monocular Visual Odometry: Mathematical Foundations, Software Architecture, Development Workflow, and a Novel Robustness Framework

Your Name / Research Group

September 9, 2026

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Why This Problem Matters	3
1.3	Report Roadmap	4
2	Development Workflow	4
3	System Inputs and Deliverables	6
3.1	Inputs	6
3.1.1	Image Sequence	6
3.1.2	Intrinsic Camera Matrix (K)	6
3.1.3	Distortion Coefficients (D)	6
3.2	Deliverables	7
3.2.1	Camera Trajectory ($T_k \in SE(3)$)	7
3.2.2	Sparse Point Cloud (M)	7
4	Methodology: Literature Review and Method Selection	7
5	In-Depth Mathematical Foundations	8
5.1	Epipolar Geometry and the Essential Matrix	8
5.1.1	Worked Numerical Example	9
5.1.2	Second Worked Example: Adding Rotation	9
5.2	Recovering the Fundamental Matrix (Pixel-Space Form)	10
5.3	Pose Recovery via SVD	10
5.4	Triangulation via the Direct Linear Transform (DLT)	10
5.5	Reprojection Error and Bundle Adjustment	10
6	Software Architecture and Codebase Design	11
6.1	Directory Structure	11
6.2	System Data Flow	11
6.3	Module-by-Module Logic	12

6.3.1	calibration.py	12
6.3.2	feature_extractor.py	12
6.3.3	pose_estimator.py	13
6.3.4	bundle_adjuster.py	13
6.3.5	visualizer.py	14
6.3.6	main.py	14
7	Error Analysis and Our Novel Approach	14
7.1	Inherent Errors in the Baseline Pipeline	14
7.2	Our Novel Mitigations	15
8	Related Work and Positioning of Novelty	15
9	Conclusion	16

Abstract

This report presents a comprehensive, self-contained treatment of Monocular Visual Odometry (MVO): the estimation of a camera’s 6-DoF trajectory from a single image stream with no external range sensor. We (1) derive every core matrix used in the pipeline element-by-element, with worked numerical examples; (2) document the complete engineering workflow that led from literature review to a working implementation, including the reasoning behind every design decision; (3) describe the software architecture in full, including the responsibility, inputs, outputs, and internal algorithmic logic of every source file and how the files communicate; (4) formally characterize the two dominant monocular failure modes – unobservable scale and dynamic-object contamination – and present our two novel mitigations, an adaptive Lowe’s-ratio threshold and epipolar-distance-based dynamic outlier rejection; and (5) position the work relative to the published literature to establish a concrete axis of novelty. The report is written so that every mathematical object is expanded in full (no matrix or vector is left opaque), so that a reader unfamiliar with epipolar geometry can follow every step.

1 Introduction

1.1 Problem Statement

Visual Odometry (VO) is the task of estimating the ego-motion of a camera-carrying agent using *only* the sequence of images it captures, without wheel encoders, GPS, or an IMU. Formally, given an ordered sequence of images $\{I_0, I_1, \dots, I_N\}$, VO seeks to recover the sequence of rigid-body poses $\{T_0, T_1, \dots, T_N\} \in SE(3)$ that describe how the camera moved between consecutive frames, and (optionally) a sparse 3D reconstruction of the scene that was used to obtain that estimate.

Monocular VO restricts the sensor to a *single* camera. This single restriction is the source of almost every difficulty this report addresses:

- A single image is a projective (bearing-only) measurement: every 3D point along a ray through the camera centre projects to the same pixel. Depth is fundamentally unobservable from one image.
- Because depth is unobservable, translation between two frames can only be recovered up to an unknown positive scalar (*scale ambiguity*). Two scenes related by a global scale factor produce identical images.
- Because there is no external metric reference (no baseline, as in stereo, and no depth sensor, as in RGB-D), any small per-frame scale error compounds multiplicatively over the trajectory, producing *scale drift*.

1.2 Why This Problem Matters

MVO is the core perception primitive behind monocular SLAM, drone navigation in GPS-denied environments, AR/VR head tracking, and low-cost mobile robotics, precisely because a single camera is cheaper, lighter, and lower-power than stereo rigs, LiDAR, or RGB-D sensors. The price of that simplicity is the scale and robustness problems analysed in Section 7.

1.3 Report Roadmap

Section 2 lays out the actual chronological workflow we followed, from literature survey to the final implementation, and the reasoning at each decision point. Section 3 formally specifies every input and deliverable of the system. Section 4 reviews the three major families of VO algorithms and justifies our selection. Section 5 derives, expands, and numerically demonstrates every mathematical object in the pipeline. Section 6 documents the full software architecture: directory layout, data flow, and the internal logic of every file. Section 7 analyses the two dominant error sources and presents our novel mitigations. Section 8 positions the work against the published literature. Section 9 concludes.

2 Development Workflow

Unlike a report that only presents a finished result, this section documents the actual sequence of engineering and research steps taken, in the order they were taken, together with the reasoning behind each transition. This is the process by which the mathematics in Section 5 and the code in Section 6 came to exist.

Step 1. Problem scoping. We defined the exact task: recover a metric-scale-free camera trajectory and a sparse 3D map from an uncalibrated or pre-calibrated monocular RGB sequence, in as close to real time as a single-threaded Python/OpenCV pipeline allows.

Step 2. Literature review. Before writing any code we surveyed the three dominant algorithmic families used in the VO/SLAM literature (direct, learning-based, feature-based) — see Section 4 for the comparison table. We read foundational papers (Nistér’s five-point algorithm, Mur-Artal et al.’s ORB-SLAM, Engel et al.’s DSO and LSD-SLAM, Forster et al.’s SVO) to understand what had already been tried, which failure modes were already documented (scale drift, dynamic objects, rolling shutter, photometric calibration), and which of those failure modes were still open problems worth targeting.

Step 3. Method selection. Based on the review, we selected the feature-based (indirect) paradigm as our foundation, since it offered the best robustness-to-implementation-complexity ratio and the clearest mathematical pipeline to build novel modifications on top of (see Section 4 for the full justification and the rejected alternatives).

Step 4. Mathematical groundwork. With the paradigm fixed, we derived the mathematical pipeline from first principles: the pinhole camera model and intrinsic matrix K (Section 3.1.2), the epipolar constraint and Essential matrix E (Section 5.1), pose recovery from E via SVD (Section 5.3), triangulation via DLT (Section 5.4), and the bundle-adjustment cost function (Section 5.5). Each formula was validated by hand with a toy numerical example (pure lateral translation, zero rotation) before being trusted in code — these worked examples are reproduced in Section 5.

Step 5. Offline calibration. We implemented `calibration.py` first, since every downstream module depends on a correct K and distortion vector D . This

was validated in isolation (checkerboard reprojection error < 0.5 px) before any odometry code was written.

- Step 6. Front-end (per-frame perception).** We implemented `feature_extractor.py` (undistortion, pyramiding, FAST detection, BRIEF description) and tested it independently by visually verifying keypoint stability across consecutive frames.
- Step 7. Two-view geometry.** We implemented the matching, RANSAC + five-point Essential-matrix estimation, and SVD-based pose recovery inside `pose_estimator.py` validating each stage against the hand-derived toy example from Step 4 before integrating with real image data.
- Step 8. Triangulation and map initialisation.** We extended `pose_estimator.py` with DLT triangulation, and cross-checked the recovered 3D points against approximate real-world distances (e.g., known checkerboard square size) to catch scale/sign errors early.
- Step 9. Backend optimisation.** We implemented `bundle_adjuster.py` using `g2o`, connecting it to the front end via a shared in-memory pose/map-point buffer maintained by `main.py`. We validated it by confirming that reprojection error dropped monotonically after each optimisation call on a static test sequence.
- Step 10. System integration.** `main.py` was written last, once every module had been validated in isolation, to orchestrate the full per-frame loop, manage the state machine (initialisation / tracking / relocalisation), and own the global trajectory and map buffers.
- Step 11. Visualisation.** `visualizer.py` was added purely for debugging and qualitative evaluation, running on a separate process so that OpenGL rendering never blocked the tracking loop.
- Step 12. Baseline error analysis.** With the full pipeline running end-to-end, we ran it on real sequences and catalogued the two catastrophic error modes actually observed: monotonic scale drift over long sequences, and severe rotation/translation corruption whenever a large moving object (e.g., a car) dominated the frame. This is documented in Section 7.
- Step 13. Targeted novelty.** Only after the baseline failure modes were empirically confirmed did we design targeted fixes: the adaptive Lowe’s-ratio threshold and the epipolar-distance dynamic-object filter (Section 7.2), both implemented as additions to `pose_estimator.py` rather than as a redesign of the pipeline, to keep the change isolated and independently ablatable.
- Step 14. Comparative positioning.** Finally, we compared our modified pipeline’s qualitative and quantitative behaviour against the literature baselines identified in Step 2, to state precisely what is novel about this work and what is not (Section 8).

3 System Inputs and Deliverables

3.1 Inputs

3.1.1 Image Sequence

An ordered stream of frames I_k , $k = 0, \dots, N$. For a grayscale frame, $I_k \in \mathbb{R}^{H \times W}$, i.e. a matrix of H rows and W columns of intensity values (typically 0–255 for 8-bit images). Colour frames are converted to grayscale before feature extraction, since FAST/BRIEF operate on intensity, not colour.

3.1.2 Intrinsic Camera Matrix (K)

The pinhole camera model maps a 3D point in the camera’s reference frame, $X_c = [X, Y, Z]^T$, to a 2D pixel coordinate $u = [u, v]^T$ via

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}, \quad K = \begin{bmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (1)$$

where $\lambda = Z$ is the depth (the homogeneous scale factor). Expanding every entry of K :

- $f_x = f/s_x$ and $f_y = f/s_y$: the focal length f (in metres) divided by the physical width/height of a single pixel sensor element (s_x, s_y , in metres/pixel). These convert a physical focal length into pixel units, and are generally unequal only if pixels are non-square.
- c_x, c_y : the pixel coordinates of the principal point, i.e. where the optical axis intersects the image plane. Ideally this is the image centre, but manufacturing tolerances shift it slightly.
- γ : the skew coefficient, non-zero only if the sensor’s row and column axes are not exactly perpendicular. For essentially all modern CMOS/CCD sensors $\gamma = 0$.

Expanding the projection explicitly:

$$u = f_x \frac{X}{Z} + \gamma \frac{Y}{Z} + c_x, \quad v = f_y \frac{Y}{Z} + c_y \quad (2)$$

This is why depth cannot be recovered from a single view: u and v depend only on the *ratios* X/Z and Y/Z , so scaling (X, Y, Z) by any constant $\alpha > 0$ produces the identical pixel coordinate. This single fact is the root cause of the scale ambiguity problem discussed throughout this report.

3.1.3 Distortion Coefficients (D)

Real lenses are not ideal pinholes; they introduce radial and tangential distortion:

$$D = [k_1 \quad k_2 \quad p_1 \quad p_2 \quad k_3]^T \quad (3)$$

With normalized image coordinates (x, y) and $r^2 = x^2 + y^2$, the corrected (undistorted) coordinates are

$$x_{\text{corr}} = x (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2 (r^2 + 2x^2) \quad (4)$$

$$y_{\text{corr}} = y (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1 (r^2 + 2y^2) + 2p_2 xy \quad (5)$$

k_1, k_2, k_3 model radial (barrel/pincushion) distortion; p_1, p_2 model tangential distortion caused by imperfect lens-sensor alignment. Every incoming frame is corrected with these coefficients before any feature is extracted (Section 6.3.2), otherwise the epipolar constraint of Section 5.1 does not hold exactly.

3.2 Deliverables

3.2.1 Camera Trajectory ($T_k \in SE(3)$)

Every pose is a 4×4 homogeneous transformation:

$$T_k = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} R & t \\ \mathbf{0}^T & 1 \end{bmatrix}, \quad R \in SO(3), t \in \mathbb{R}^3 \quad (6)$$

The rotation block R is a 3×3 orthonormal matrix ($R^T R = I$, $\det(R) = 1$) with 9 numbers but only 3 true degrees of freedom (any rotation can be parameterised by, e.g., a 3-vector of axis-angle or a unit quaternion — R is simply the redundant matrix form used for computation). $t = [t_x, t_y, t_z]^T$ is the translation of the camera centre. Applying T_k to a homogeneous point $[X, Y, Z, 1]^T$ in the world frame yields its coordinates in the frame of camera k .

3.2.2 Sparse Point Cloud (M)

A set $M = \{X_i\}_{i=1}^P$ of 3D landmarks, $X_i = [X, Y, Z]^T \in \mathbb{R}^3$, each corresponding to a physical surface point that was successfully tracked and triangulated across at least two views (Section 5.4). This is a *sparse* reconstruction (hundreds to a few thousand points), not a dense surface model.

4 Methodology: Literature Review and Method Selection

We compared the three dominant algorithmic families before committing to an approach.

Family	Principle	Reason for rejection / selection
Direct methods (e.g. LSD-SLAM, DSO)	Minimise a <i>photometric</i> error directly on pixel intensities, without extracting explicit features.	Rejected. Assumes photometric consistency (brightness constancy) between frames; degrades quickly under auto-exposure changes, specular highlights, or dynamic lighting. Also requires careful photometric calibration to perform well, adding an extra offline dependency.

Family	Principle	Reason for rejection / selection
Learning-based methods (e.g. DeepVO, UnDeepVO)	Train a neural network (typically a CNN or CNN-RNN) to regress T_k directly from image pairs/sequences.	Rejected. Strong dependence on the training distribution; generalises poorly to camera intrinsics, environments, or motion profiles unseen in training. Also opaque: harder to attribute a specific failure to a specific mathematical stage, which conflicts with our goal of building a well-understood, ablatable pipeline.
Feature-based / indirect methods (e.g. ORB-SLAM family)	Extract sparse, repeatable key-points and descriptors, match them across frames, and solve classical multi-view geometry problems (Essential matrix, PnP, bundle adjustment).	Chosen. Well-understood closed-form and iterative solutions exist for every stage; robust to moderate lighting change (descriptor-based matching, not raw photometric residuals); every stage is independently testable, which is what allowed us to isolate and fix specific failure modes (Section 7.2) instead of retraining or redesigning an end-to-end system.

Table 1: Comparison of VO algorithm families and the justification for our selection.

5 In-Depth Mathematical Foundations

5.1 Epipolar Geometry and the Essential Matrix

Consider two consecutive frames I_1, I_2 and a static 3D point X observed at normalized image coordinates x_1 in I_1 and x_2 in I_2 (normalized meaning K^{-1} has already been applied, so these are coordinates on the unit-depth plane, not pixels). The two views satisfy the **epipolar constraint**:

$$x_2^T E x_1 = 0, \quad E = [t]_{\times} R \quad (7)$$

E is the 3×3 **Essential matrix**, encoding the relative rotation R and translation direction t between the two camera poses (up to the unknown scale of t — again, the scale ambiguity of Section 3.1.2). $[t]_{\times}$ is the skew-symmetric *cross-product matrix* of $t = [t_x, t_y, t_z]^T$, defined so that $[t]_{\times} v = t \times v$ for any vector v :

$$[t]_{\times} = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \quad (8)$$

Note $[t]_{\times}$ is always singular ($\det = 0$) and antisymmetric ($[t]_{\times}^T = -[t]_{\times}$); this is why E has rank 2, a fact exploited when enforcing the essential-matrix constraints after estimation (the two non-zero singular values of E are made equal by construction/projection onto the essential manifold).

5.1.1 Worked Numerical Example

Let the camera translate purely along $+x$ by one unit, $t = [1, 0, 0]^T$, with zero rotation, $R = I_3$. Then

$$[t]_{\times} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}, \quad E = [t]_{\times} R = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \quad (9)$$

Take a 3D point $X = [0, 0, 5]^T$ in the first camera frame. Its normalized projection is $x_1 = [X/Z, Y/Z, 1]^T = [0, 0, 1]^T$. After the camera moves $+1$ along x , the same point expressed in the second camera frame is $X_2 = X - t = [-1, 0, 5]^T$ (subtracting because the camera, not the point, moved), giving normalized projection

$$x_2 = \left[\frac{-1}{5}, \frac{0}{5}, 1 \right]^T = [-0.2, 0, 1]^T \quad (10)$$

Verifying the constraint:

$$Ex_1 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix}, \quad x_2^T(Ex_1) = [-0.2 \quad 0 \quad 1] \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} = 0 \quad (11)$$

confirming the two projections lie on corresponding epipolar lines, as required.

5.1.2 Second Worked Example: Adding Rotation

To illustrate the role of R , add a small yaw rotation of $\theta = 10^\circ \approx 0.1745$ rad about the camera's y -axis, keeping $t = [1, 0, 0]^T$:

$$R = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix} \approx \begin{bmatrix} 0.9848 & 0 & 0.1736 \\ 0 & 1 & 0 \\ -0.1736 & 0 & 0.9848 \end{bmatrix} \quad (12)$$

Then

$$E = [t]_{\times} R = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0.9848 & 0 & 0.1736 \\ 0 & 1 & 0 \\ -0.1736 & 0 & 0.9848 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0.1736 & 0 & -0.9848 \\ 0 & 1 & 0 \end{bmatrix} \quad (13)$$

(Each row of E is obtained by taking the corresponding row of $[t]_{\times}$ and using it as weights over the rows of R .) This shows explicitly how rotation *mixes into* the Essential matrix multiplicatively, which is why decoupling R and t from a noisy estimate of E (Section 5.3) is a non-trivial decomposition problem, not a simple matrix split.

5.2 Recovering the Fundamental Matrix (Pixel-Space Form)

In practice, correspondences are first found in raw pixel coordinates u_1, u_2 , not normalized coordinates. The **Fundamental matrix** F relates pixel coordinates directly:

$$u_2^T F u_1 = 0, \quad F = K^{-T} E K^{-1}, \quad E = K^T F K \quad (14)$$

F absorbs the (unknown, if uncalibrated) intrinsics into the constraint; since our system calibrates the camera offline (Section 6.3.1), we recover E directly using the calibrated five-point algorithm rather than estimating F with the (uncalibrated) eight-point algorithm and converting.

5.3 Pose Recovery via SVD

Given an estimated E (from RANSAC + the five-point algorithm, Section 6.3.3), R and t are recovered via Singular Value Decomposition:

$$E = U \Sigma V^T, \quad \Sigma = \text{diag}(\sigma, \sigma, 0) \quad (15)$$

Define the fixed rotation matrix

$$W = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (16)$$

The four algebraically valid decompositions are

$$R \in \{U W V^T, U W^T V^T\}, \quad t \in \{+u_3, -u_3\} \quad (17)$$

where u_3 is the third column of U (the left null vector of E , i.e. the translation direction up to sign and unknown scale). This gives $2 \times 2 = 4$ candidate (R, t) pairs; exactly one places the triangulated 3D points in front of *both* cameras (positive depth, the **cheirality check**), and that pair is selected as the true relative pose.

5.4 Triangulation via the Direct Linear Transform (DLT)

Once (R, t) is fixed, each matched pixel pair (u_1, u_2) is triangulated into a 3D point X . With projection matrices $P_1 = K[I \mid 0]$ and $P_2 = K[R \mid t]$, each view contributes two linear equations in the unknown homogeneous $X = [X, Y, Z, 1]^T$:

$$u(P^{(3)}X) - P^{(1)}X = 0, \quad v(P^{(3)}X) - P^{(2)}X = 0 \quad (18)$$

where $P^{(i)}$ denotes row i of P . Stacking the two equations from each of the two views gives a 4×4 homogeneous linear system $AX = 0$; X is taken as the right singular vector of A corresponding to the smallest singular value (again via SVD), then dehomogenised by dividing through by its fourth coordinate.

5.5 Reprojection Error and Bundle Adjustment

Bundle Adjustment (BA) jointly refines all recent poses $\{T_k\}$ and map points $\{X_i\}$ by minimising the total **reprojection error**:

$$e_{i,k} = u_{i,k} - \pi(K, T_k X_i) \quad (19)$$

where $u_{i,k}$ is the pixel actually observed for landmark i in frame k , and π projects the 3D point (transformed into camera k 's frame by T_k , then projected by K as in Section 3.1.2) back into the image. The full non-linear least-squares objective is

$$\{T_k^*, X_i^*\} = \arg \min_{T, X} \sum_k \sum_i \rho(e_{i,k}^T \Sigma^{-1} e_{i,k}) \quad (20)$$

- Σ^{-1} is the inverse measurement-noise covariance (an information weighting — more confident/precise detections are weighted more heavily).
- $\rho(\cdot)$ is a robust loss (e.g., Huber), which caps the influence of any single large residual so that a handful of bad matches (including undetected dynamic-object outliers, Section 7) cannot dominate the optimisation.

This is solved iteratively (Gauss-Newton / Levenberg-Marquardt) inside `bundle_adjuster.py` using the `g2o` graph-optimisation library (Section 6.3.4).

6 Software Architecture and Codebase Design

6.1 Directory Structure

```
mvo_workspace/
data/
  image_sequence/      # Raw input frames (.png, .jpg)
  camera_params.yaml   # Serialized intrinsic matrix K and distortion D
src/
  __init__.py
  main.py              # Central orchestrator + state machine
  calibration.py       # Offline intrinsic calibration
  feature_extractor.py  # Undistortion, pyramids, FAST + BRIEF
  pose_estimator.py    # Matching, RANSAC+5-point, SVD pose, DLT triangulation,
                      #   + our novel adaptive-ratio and epipolar-distance fil
  bundle_adjuster.py   # g2o-based backend graph optimisation
  visualizer.py        # Real-time 3D rendering (separate process)
requirements.txt
```

6.2 System Data Flow

Figure (described textually below, since this is a document, not a live diagram) shows the flow of data:

1. **Offline, once:** `calibration.py` consumes checkerboard images from `data/` and writes K , D into `camera_params.yaml`.
2. **At boot:** `main.py` loads `camera_params.yaml` and instantiates one object each of `FeatureExtractor`, `PoseEstimator`, `BundleAdjuster`, `Visualizer`.
3. **Per frame I_k :** `main.py` $\rightarrow$ `feature_extractor.py`: raw frame in $\rightarrow$ (keypoint array $N \times 2$, descriptor array $N \times 256$) out.

4. `main.py` $\rightarrow$ `pose_estimator.py`: current + previous frame's (keypoints, descriptors) in $\rightarrow$ relative pose (R, t) and newly triangulated 3D points out.
5. `main.py` composes the relative pose onto the running trajectory ($T_k = T_{k-1} \cdot [R | t]$ in homogeneous form) and appends new map points to the global map.
6. **Periodically (every M keyframes):** `main.py` $\rightarrow$ `bundle_adjuster.py`: a local window of poses + map points in $\rightarrow$ corrected poses + map points out, which overwrite the global buffers.
7. **Continuously, on a separate process:** `main.py` $\rightarrow$ `visualizer.py`: streamed copies of the trajectory and map for rendering, decoupled so that rendering never blocks tracking.

This makes `main.py` the single owner of shared state (the global trajectory list and map-point dictionary); every other module is a stateless (or self-contained-state) function of its inputs, which is what keeps each module independently testable, as described in Section 2.

6.3 Module-by-Module Logic

6.3.1 `calibration.py`

Purpose: produce K and D (Section 3.1.2).

Inputs: a directory of checkerboard images (known real-world square size).

Outputs: `camera_params.yaml` containing K and D .

Core logic: for every image, `cv2.findChessboardCorners` locates the checkerboard's 2D grid intersections. Because the same grid's 3D coordinates in its own reference frame are known exactly (planar, regularly spaced, $Z = 0$), each image supplies a set of 2D $\leftrightarrow$ 3D correspondences. Zhang's method poses this as a non-linear least-squares problem over all images simultaneously, solved with Levenberg-Marquardt, minimising the total reprojection error (the same error form as Section 5.5, but here the "pose" being solved for is the checkerboard's pose relative to the camera in each image, and the free parameters are K and D shared across all images). This module runs once, offline, before any odometry.

6.3.2 `feature_extractor.py`

Purpose: turn a raw pixel matrix I_k into trackable, matchable numerical vectors.

Inputs: raw frame I_k , K , D .

Outputs: keypoint array ($N \times 2$ pixel coordinates), descriptor array ($N \times 256$ binary vectors).

Core logic (in execution order):

1. **Undistortion:** applies the inverse of the distortion model in Section 3.1.2 using K, D , so that straight lines in the world are straight in the image and the epipolar constraint of Section 5.1 holds.
2. **Image pyramids:** the frame is repeatedly downsampled (e.g. by a factor of $\sqrt{2}$ per level, 4-8 levels) so the same physical corner is detectable whether the camera is close to or far from it (scale invariance) — a feature seen at level ℓ has effective scale $2^{\ell/2}$ relative to the original.

3. **FAST detection:** for each candidate pixel, intensities on a 16-pixel Bresenham circle around it are compared to the centre pixel $\pm$ a threshold t_{FAST} ; the pixel is a corner if ≥ 12 contiguous circle pixels are all brighter or all darker than the centre by more than t_{FAST} .
4. **BRIEF description:** around each detected corner, 256 pairs of nearby pixels (from a fixed, pre-defined sampling pattern) are compared; each comparison contributes one bit, producing a compact 256-bit binary descriptor that is fast to compare via Hamming distance.

Returns the $N \times 2$ keypoint-coordinate array and $N \times 256$ descriptor array to `main.py`.

6.3.3 pose_estimator.py

Purpose: compute the relative pose (R, t) between consecutive frames and triangulate new 3D points; also hosts our two novel robustness additions (Section 7.2).

Inputs: keypoints/descriptors for I_{k-1} and I_k .

Outputs: relative pose (R, t) , newly triangulated points X_i , and a per-point inlier/dynamic-outlier flag.

Core logic (in execution order):

1. **Brute-force matching:** every descriptor in frame $k-1$ is compared, via Hamming distance, to every descriptor in frame k ; the closest and second-closest matches are kept for each keypoint.
2. **Adaptive Lowe’s ratio test (novel, Section 7.2):** a match is kept only if $d_{\text{closest}}/d_{\text{second-closest}} < \tau$, where τ is adjusted per-frame based on the frame’s motion-blur estimate.
3. **Essential matrix estimation:** the surviving matches, normalized by K^{-1} , are passed to the five-point algorithm wrapped in a RANSAC loop, which repeatedly samples minimal sets of 5 correspondences, computes a candidate E , and counts inliers satisfying $x_2^T E x_1 \approx 0$ within a pixel-distance tolerance; the E with the most inliers is kept.
4. **Pose recovery:** SVD-based decomposition of E into (R, t) as in Section 5.3, with the cheirality check to disambiguate the four candidate solutions.
5. **Strict epipolar-distance dynamic filtering (novel, Section 7.2):** every inlier’s perpendicular distance to its epipolar line is recomputed using the *final* E and compared to a strict threshold; points exceeding it are flagged as dynamic and excluded from triangulation and from the Bundle Adjustment graph.
6. **Triangulation:** DLT (Section 5.4) is applied to every surviving static inlier to produce new map points.

6.3.4 bundle_adjuster.py

Purpose: suppress accumulated drift over a local window of frames (Section 5.5).

Inputs: a local window of recent poses $\{T_k\}$ and map points $\{X_i\}$ (excluding anything flagged dynamic by `pose_estimator.py`).

Outputs: corrected poses and map points.

Core logic: builds a bipartite factor graph in `g2o` — pose vertices in $SE(3)$, landmark vertices in $\mathbb{R}^3$, and reprojection-error edges connecting each landmark to every pose that observed it. `g2o`’s Levenberg-Marquardt solver iteratively perturbs every vertex to minimise the summed robustified reprojection error (Section 5.5) until convergence, then the corrected values are written back to `main.py`’s global buffers.

6.3.5 visualizer.py

Purpose: real-time qualitative feedback (trajectory + map) for debugging and evaluation.

Inputs: streamed copies of the global trajectory and map.

Core logic: because OpenGL rendering would otherwise contend with the Python GIL and stall the main tracking loop, this module is spawned as a separate daemon process (via `multiprocessing`) and communicates with `main.py` over an inter-process queue. It uses `Pangolin` to draw camera frustums along the estimated trajectory and to plot sparse map points as OpenGL points, updated every frame.

6.3.6 main.py

Purpose: the central execution loop, state manager, and sole owner of global shared state.

Core logic: initialises all module objects with K, D ; enters a per-frame `while` loop implementing a three-state machine (**Initialisation** — bootstrap the first two keyframes and initial map; **Tracking** — the steady-state per-frame loop of Section 4.2; **Relocalisation** — triggered when the inlier count from `pose_estimator.py` drops below a threshold, indicating tracking loss, at which point the system attempts to re-match the current frame against recent keyframes rather than the immediately previous frame). `main.py` also owns and mutates the global trajectory list and map-point dictionary that every other module reads from or writes to.

7 Error Analysis and Our Novel Approach

7.1 Inherent Errors in the Baseline Pipeline

1. **Scale drift.** As shown in Section 5.3, t is recovered from the null vector u_3 of E , which is only defined up to sign and is implicitly normalised ($\|t\| = 1$) by the SVD. There is no mechanism in the base pipeline that ties this per-frame unit-norm t to a consistent real-world scale across frames; over hundreds/thousands of frames, small per-frame scale inconsistencies accumulate multiplicatively, warping the trajectory (most visibly as an incorrect turning radius or an incorrect final loop-closure gap).
2. **Dynamic-object contamination.** Standard RANSAC assumes that *most* correspondences are geometrically consistent with a single rigid motion. When a large moving object (e.g., a car crossing the frame) supplies enough self-consistent correspondences, RANSAC can lock onto the wrong motion hypothesis or dilute the inlier set, corrupting the recovered R (rotation errors are visually the most damaging, since they compound directionally across the whole subsequent trajectory).

7.2 Our Novel Mitigations

Both additions live inside `pose_estimator.py` (Section 6.3.3) as isolated, ablatable stages rather than as a pipeline redesign, consistent with the workflow philosophy in Section 2.

1. **Adaptive Lowe’s-ratio threshold.** The classical Lowe’s ratio test uses a single fixed threshold τ (commonly 0.7–0.8) for every frame. We instead compute a per-frame motion-blur score as the variance of the Laplacian of the (grayscale) frame, $\text{Var}(\nabla^2 I_k)$ — a low variance indicates a blurred, low-detail frame where descriptors are inherently less discriminative. τ is tightened (lowered) as this variance drops, which rejects more borderline matches exactly when they are least trustworthy, rather than admitting a fixed proportion of matches regardless of frame quality.
2. **Strict epipolar-distance dynamic filtering.** After RANSAC converges on a final E , we recompute, for every inlier, the perpendicular pixel distance from x_2 to its predicted epipolar line $l_2 = Ex_1$ (line in the form $[a, b, c]$ with distance $|au_2 + bv_2 + c|/\sqrt{a^2 + b^2}$ after de-normalising by K). Points on a rigidly moving object satisfy a *different* epipolar geometry than the dominant (static-background) motion and will systematically show larger residual distances than sensor noise alone would produce; a strict threshold flags and permanently removes such points before they can enter triangulation (Section 5.4) or the Bundle Adjustment graph (Section 5.5), preventing them from corrupting either the current pose estimate or the backend map.

Because scale drift and dynamic contamination are independent failure modes, they are addressed independently: the ratio-test change improves match quality *before* E is even estimated, while the epipolar-distance filter acts *after* E is estimated, cleaning the map/BA input. Neither mitigation solves the scale-drift problem itself (which fundamentally requires an external metric reference such as an IMU, stereo baseline, or known object size, and is out of scope for a purely monocular system); the report is explicit about this limitation rather than overstating what a monocular-only fix can achieve.

8 Related Work and Positioning of Novelty

The literature already documents both failure modes we target (see the reference list below): scale drift is characterised in scale-drift-aware SLAM work, and dynamic-environment robustness is surveyed broadly in the VSLAM-in-dynamic-environments literature. Our contribution is not the discovery of these problems, but two lightweight, self-contained modifications to the classical indirect pipeline’s front end — an adaptive matching threshold driven by a motion-blur proxy, and a post-RANSAC epipolar-distance dynamic filter — that require no additional sensors, no learned model, and no architectural change to the rest of the pipeline. When you populate the empirical section of your own paper, the concrete axis of comparison should be: (a) trajectory accuracy (Absolute Trajectory Error / Relative Pose Error) against ORB-SLAM2/ORB-SLAM3 and DSO on a standard benchmark (e.g., KITTI or TUM RGB-D/TUM-Mono) with and without your two modifications (an ablation), and (b) qualitative/quantitative robustness specifically on sequences containing moving objects, since that is where the epipolar-distance filter should show its largest relative improvement.

9 Conclusion

This report has documented, in full mathematical and architectural detail, a feature-based monocular visual odometry pipeline: the pinhole/intrinsic model, the epipolar constraint and Essential matrix with two fully worked numerical examples, SVD-based pose recovery, DLT triangulation, and graph-based bundle adjustment; the complete software architecture including every file’s inputs, outputs, and internal logic and how they connect through `main.py`’s central state; the actual chronological engineering workflow from literature review through integration; and a precise characterisation of two baseline failure modes together with two isolated, novel mitigations. Section 8 identifies the concrete empirical comparisons needed to substantiate novelty claims against the published literature once benchmark results are available.